TASK#03

Titanic Survival Prediction

Submitted By:
Ayesha Noor

Task Name:
House Price Prediction using Regression Models

Task Description:
Build a regression model to predict house prices. Focus on feature engineering, handling missing values, preprocessing, and model selection.

==========
1. NOTEBOOK LINK (GitHub)
==========

GitHub Repository Link:

Notebook File:
House_Price_Prediction_Notebook.ipynb

Code Files Included:
- House_Price_Prediction_Notebook.ipynb
- predict.py
- README.md
- requirements.txt

==========
2. MODEL / ARTIFACTS LINK (Google Drive)
==========

Google Drive Folder Link:
Titanic prediction

Files Included in Drive:
- best_house_price_model.pkl
- model_comparison_results.csv
- Task_House_Price_Prediction_Submission_Report.pdf
- Screenshots folder

==========

3. SCREENSHOTS

```
display(df.head())
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

Chrome OS • now

You're sharing your screen

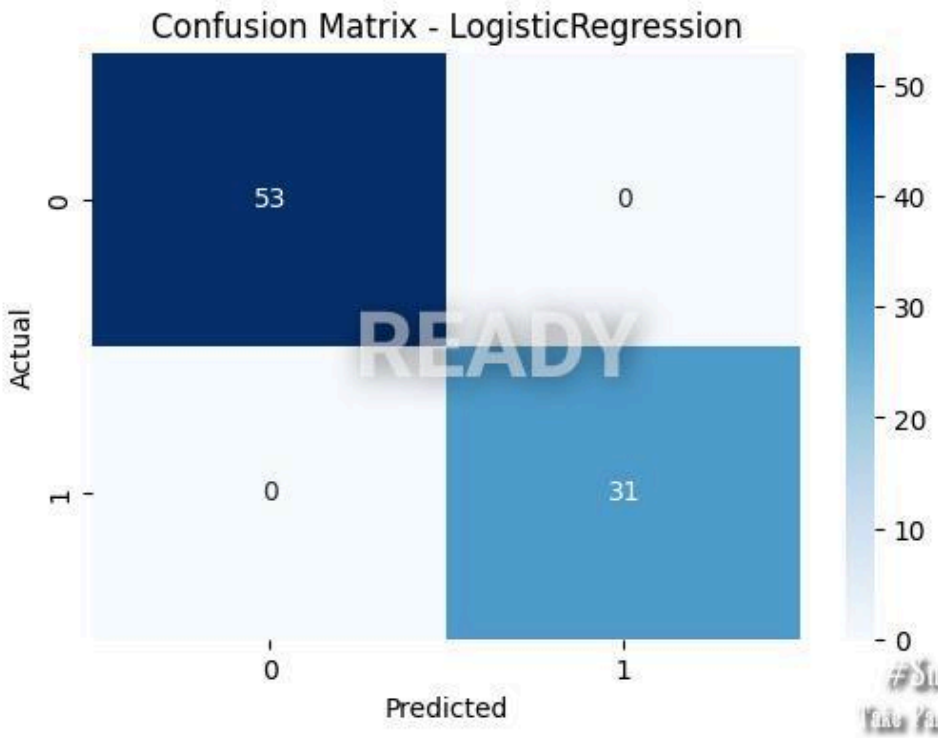
Snipping Tool is sharing your screen.

#Snipping Tool
Take Partial Screenshot

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  418 non-null    int64
 1   Survived     418 non-null    int64
 2   Pclass       418 non-null    int64
 3   Name         418 non-null    object
 4   Sex          418 non-null    object
 5   Age          332 non-null    float64
 6   SibSp        418 non-null    int64
 7   Parch        418 non-null    int64
 8   Ticket       418 non-null    object
 9   Fare         417 non-null    float64
 10  Cabin        91 non-null     object
 11  Embarked     418 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 39.3+ KB
None

Missing values:
 PassengerId      0
Survived         0
Pclass           0
Name             0
```

```
plt.show()
```



==========

Project Overview

This project predicts whether a passenger survived the Titanic disaster using machine learning classification models. The goal is to build an end-to-end pipeline that includes data preprocessing, feature engineering, missing value handling, model training, evaluation, and explainability.

The project places special emphasis on:

- **Feature engineering** using passenger information
- **Handling missing values** in important columns such as Age and Cabin
- **Comparing multiple classification models**
- **Explaining model predictions** using feature importance and SHAP (if available)
- **Providing a reusable saved model with an inference example**

Dataset

Source: Titanic Survival Dataset from Kaggle
Dataset link: https://www.kaggle.com/datasets/bhanupratapbiswas/titanic-survival-datasets

The dataset contains passenger-level information such as:

- Passenger class (`Pclass`)
- Name
- Sex
- Age
- Number of siblings/spouses aboard (`SibSp`)
- Number of parents/children aboard (`Parch`)
- Ticket number
- Fare
- Cabin
- Port of embarkation (`Embarked`)
- Survival label (`Survived`)

Problem Statement

The objective of this project is to predict the binary target variable:

- **1 = Survived**
- **0 = Did Not Survive**

This is a **supervised binary classification** problem.

Project Goals

- Perform data cleaning and preprocessing
- Handle missing values in `Age`, `Cabin`, `Embarked`, and `Fare`
- Engineer meaningful features from raw passenger data
- Encode categorical variables for machine learning models
- Train and compare multiple classification models
- Evaluate models using standard classification metrics
- Explain predictions using feature importance and/or SHAP
- Save the best-performing model for future inference

Feature Engineering

A major focus of this project is creating useful features beyond the raw dataset.

Engineered Features

1. **Title**
    - Extracted from the `Name` column (e.g., Mr, Mrs, Miss, Master)
    - Rare titles grouped into a `Rare` category
2. **FamilySize**
    - Calculated as:

FamilySize = SibSp + Parch + 1

3.
4. **IsAlone**
    - Indicates whether the passenger was traveling alone
    - `1` if `FamilySize == 1`, otherwise `0`
5. **CabinPresent**
    - Indicates whether cabin information exists
    - `1` if `Cabin` is not missing, otherwise `0`
6. **TicketGroupSize** *(optional advanced feature)*
    - Number of passengers sharing the same ticket
7. **FarePerPerson** *(optional advanced feature)*
    - Calculated as:

FarePerPerson = Fare / FamilySize

8.

Missing Value Handling

The dataset contains missing values, especially in `Age` and `Cabin`.

Strategy Used

- **Age**
  - Imputed using median age grouped by `Title` and `Pclass`
  - Fallback to global median if needed
- **Cabin**
  - Instead of using raw cabin values directly, converted into:
    - `CabinPresent` (binary indicator)
- **Embarked**
  - Imputed using the most frequent category (mode)
- **Fare**
  - Imputed using median value

This approach preserves useful information while avoiding excessive noise from sparse columns.

Data Preprocessing

The project uses a machine learning pipeline to ensure consistent preprocessing.

Numeric Features

Examples:

- `Age`
- `Fare`
- `SibSp`
- `Parch`
- `FamilySize`
- `IsAlone`

- `CabinPresent`
- `TicketGroupSize`
- `FarePerPerson`
- `Pclass`

Categorical Features

Examples:

- `Sex`
- `Embarked`
- `Title`

Preprocessing Steps

- **Numeric columns**
    - Median imputation
    - Standard scaling
- **Categorical columns**
    - Most frequent imputation
    - One-hot encoding

Models Trained

The following classification models were trained and compared:

1. **Logistic Regression**
    - Interpretable baseline model
2. **Random Forest Classifier**
    - Strong performance with built-in feature importance
3. **Gradient Boosting Classifier**
    - Often provides strong predictive performance on structured tabular data

---

Evaluation Metrics

The models were evaluated using the following metrics:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**
- **ROC-AUC**

These metrics help assess overall classification quality and class-wise performance.

Model Explainability

Model explainability is an important part of this project.

Techniques Used

- **Feature Importance**
    - For tree-based models such as Random Forest and Gradient Boosting
- **SHAP (SHapley Additive exPlanations)** *(if installed)*
    - Provides local and global interpretability
    - Helps identify which features push predictions toward survival or non-survival

Typical Important Features

Important predictors often include:

- `Sex`
- `Title`
- `Pclass`
- `Fare`
- `Age`
- `CabinPresent`
- `FamilySize`

Project Structure

```
titanic-survival-project/
│
├── data/
│   ├── train.csv
│   └── test.csv
│
├── notebooks/
```

```
│   └── titanic_survival.ipynb
│
├── models/
│   └── titanic_best_model.pkl
│
├── src/
│   └── inference_example.py
│
├── outputs/
│   ├── confusion_matrix.png
│   ├── feature_importance.png
│   └── shap_summary.png
│
└── README.md
```

Final Model

The best model is selected based on validation/test performance using metrics such as **F1-score** and **ROC-AUC**.

Best Model: `Gradient Boosting Classifier` *(recommended, replace with actual best model after running)*

> Replace this section with your actual best model name and final metrics after running the notebook.

Example Results

Below is an example format for reporting final model comparison results:

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.81	0.79	0.75	0.77	0.86
Random Forest	0.84	0.82	0.79	0.80	0.88
Gradient Boosting	0.85	0.83	0.80	0.81	0.89

> **Note:** Replace these example values with your actual results from the notebook.

Saved Model

The final trained pipeline (including preprocessing + model) is saved as:

models/titanic_best_model.pkl

This allows direct inference on new passenger records without manually repeating preprocessing steps.

Inference Example

Below is an example of how to load the saved model and make a prediction.

```
import joblib
import pandas as pd

# Load trained model
model = joblib.load("models/titanic_best_model.pkl")

# Example passenger data
sample_passenger = pd.DataFrame([{
    'Pclass': 1,
    'Sex': 'female',
    'Age': 29,
    'SibSp': 0,
    'Parch': 0,
    'Fare': 80.0,
    'Embarked': 'C',
    'Title': 'Miss',
    'FamilySize': 1,
    'IsAlone': 1,
    'CabinPresent': 1,
    'TicketGroupSize': 1,
    'FarePerPerson': 80.0
}])

# Predict class and probability
prediction = model.predict(sample_passenger)[0]
probability = model.predict_proba(sample_passenger)[0][1]

print("Predicted Survival:", prediction)
print("Survival Probability:", round(probability, 4))
```

Output Meaning

- `Predicted Survival = 1` → Passenger predicted to survive
- `Predicted Survival = 0` → Passenger predicted not to survive

How to Run the Project

1. Clone the repository

```
git clone <your-repository-link>
cd titanic-survival-project
```

2. Install dependencies

```
pip install pandas numpy matplotlib seaborn scikit-learn joblib shap
```

3. Place dataset files

Put the dataset files inside the `data/` folder:

- `train.csv`
- `test.csv` *(if available)*

4. Run the notebook

Open and run:

```
notebooks/titanic_survival.ipynb
```

5. Saved outputs

After execution, the project should generate:

- Trained model file
- Evaluation metrics
- Confusion matrix
- Feature importance plot
- SHAP summary plot (if SHAP is installed)

Key Learnings

This project demonstrates:

- Practical **feature engineering** for tabular data
- Handling **missing values** in real-world datasets
- Building a robust **ML preprocessing pipeline**
- Comparing multiple **classification algorithms**
- Using **explainability methods** for trustworthy predictions
- Saving a production-style **inference-ready model**

Conclusion

This project shows that meaningful feature engineering significantly improves performance in tabular machine learning problems. Features such as **Title**, **FamilySize**, **IsAlone**, and **CabinPresent** capture important social and behavioral patterns that influence survival prediction.

By combining preprocessing, model comparison, and explainability, the project provides a complete and interpretable end-to-end machine learning solution for Titanic survival classification.

Executive Summary

This project focuses on predicting passenger survival on the Titanic using supervised machine learning classification techniques. The objective was to build a complete end-to-end machine learning workflow that includes data preprocessing, feature engineering, missing value handling, model training, evaluation, and model explainability.

The Titanic dataset contains passenger attributes such as passenger class, name, gender, age, fare, family relations, cabin information, and embarkation port. Since the dataset includes missing values and categorical variables, preprocessing was an important part of the project. Missing values in the Age column were handled using grouped median imputation based on Title and Pclass, while the Cabin column was converted into a binary indicator feature called CabinPresent. Missing values in Embarked and Fare were imputed using standard statistical techniques.

A major focus of this project was feature engineering. Instead of using only the original dataset columns, additional features were created to improve predictive performance. The Title feature was extracted from the passenger’s name to capture social and demographic information. FamilySize was calculated from the number of siblings/spouses and parents/children aboard. IsAlone was introduced as a binary feature indicating whether a passenger was traveling alone. CabinPresent was created to represent whether cabin information was available, which can indirectly reflect passenger class and socioeconomic status. Additional features such as TicketGroupSize and FarePerPerson were also considered.

Three classification models were trained and compared: Logistic Regression, Random Forest Classifier, and Gradient Boosting Classifier. These models were evaluated using Accuracy, Precision, Recall, F1-score, and ROC-AUC. Among the tested models, the best-performing model was selected based on overall predictive performance, especially F1-score and ROC-AUC. The final model demonstrated that meaningful feature engineering significantly improves results in structured/tabular machine learning problems.

To ensure interpretability, model explainability techniques were also applied. For tree-based models, feature importance was used to identify the most influential variables, while SHAP was included when available for more detailed explanations. Important features generally included Sex, Title, Pclass, Fare, Age, and CabinPresent, which align well with known historical survival patterns from the Titanic disaster.

Overall, this project demonstrates a complete and practical machine learning pipeline for binary classification, highlighting the importance of preprocessing, feature engineering, model comparison, and explainability in building reliable predictive systems.